Frontend SSE Client Scaffold

Frontend SSE Client Scaffold

Overview

This document provides a complete scaffold for the **Frontend SSE (Server-Sent Events) Client**, designed for the Agent Dashboard MVP. It replaces the WebSocket client with a more resilient SSE-based event stream, ideal for human-in-the-loop workflows, long pauses, and polyglot backend architectures using Redis and a Node.js Gateway.
This scaffold integrates directly with the existing frontend event bus (routeEvent(event)), session streams, and UI components.

📁 Folder Structure

```
/frontend
  /api
    eventStreamClient.ts
  /state
    eventBus.ts
  /utils
    parseEvent.ts
```

📄 File: /api/eventStreamClient.ts

```
import { routeEvent } from "../state/eventBus";
import { parseEvent } from "../utils/parseEvent";

export class EventStreamClient {
  private eventSource: EventSource | null = null;

  connect(sessionId: string) {
    const url = `http://localhost:3001/sessions/${sessionId}/events`;

    this.eventSource = new EventSource(url);

    this.eventSource.onopen = () => {
      console.log(`SSE connected for session ${sessionId}`);
    };

    this.eventSource.onerror = (err) => {
      console.error("SSE error", err);
    };

    this.eventSource.onmessage = (msg) => {
      try {
        const event = parseEvent(msg.data);
        routeEvent(event);
      } catch (e) {
        console.error("Failed to parse SSE event", e);
      }
    };
  }

  disconnect() {
    if (this.eventSource) {
      this.eventSource.close();
      this.eventSource = null;
    }
  }
}
```

File: /utils/parseEvent.ts

```
export function parseEvent(raw: string): any {
  try {
    return JSON.parse(raw);
  } catch (e) {
    console.error("Invalid JSON from SSE", raw);
    throw e;
  }
}
```

Integration: Using the SSE Client in the Frontend

Add this to your Workspace.tsx or wherever sessions are initialized.

```
import React, { useEffect } from "react";
import { EventStreamClient } from "../api/eventStreamClient";

export function Workspace({ sessionId }: { sessionId: string }) {
  useEffect(() => {
    const client = new EventStreamClient();
    client.connect(sessionId);

    return () => client.disconnect();
  }, [sessionId]);

  return (
    <div className="workspace">
      {/* existing components */}
    </div>
  );
}
```

Gateway SSE Endpoint (Reference)

Your Node.js Gateway should expose an SSE endpoint like:

```
app.get("/sessions/:sessionId/events", (req, res) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");

  const sessionId = req.params.sessionId;

  const redisSub = redisClient.duplicate();
  redisSub.subscribe(`session:${sessionId}:events`);

  redisSub.on("message", (_, message) => {
    res.write(`data: ${message}\n\n`);
  });

  req.on("close", () => {
    redisSub.unsubscribe();
    redisSub.quit();
  });
});
```

This ensures:

- Python/Node agents publish events to Redis
- Gateway forwards them to the frontend via SSE
- Frontend routes them into the RxJS event bus

📄 Event Flow Diagram

```
Python/Node Agent → Redis Pub/Sub → Gateway SSE → Frontend EventSource →
routeEvent(event) → RxJS Bus → UI
```

🎯 Summary

This SSE client scaffold provides:

- A resilient event stream for long-running or paused workflows
- Seamless integration with the existing RxJS event bus
- A clean separation between Gateway, Redis, and frontend
- A drop-in replacement for WebSockets

It is fully aligned with the /spec contracts and ready for integration.